

Name: Aman Singh

Team 3

State Transition Tables

What is a State Transition Table?

A State Transition Table is a testing and design technique used to document how a system moves between different **states** in response to specific **events or triggers**. Each row typically captures three things: the **Current State** the system is in, the **Event/Trigger** that occurs, and the resulting **Next State** the system transitions to. This technique is especially useful for systems that have a clear lifecycle — such as an order, a job application, or a social connection — because it forces every possible transition (including edge cases like cancellations, failures, and exceptions) to be explicitly documented. The accompanying **State Transition Diagram** represents the same information visually, with states as nodes and events as labeled arrows connecting them, making the overall flow easier to follow at a glance. Together, these two artifacts are widely used in software testing to design test cases that verify valid transitions, invalid transitions, and unreachable states.

The following sections walk through five real-world examples — an e-commerce order (Flipkart), a job application (Naukri.com), a social connection (Facebook), a fashion e-commerce order (Myntra), and a food delivery order (Swiggy) — each with its state transition table and diagram.

1. Flipkart — Order Lifecycle

Flipkart's orderflow models the full journeyofane-commerce purchase, from placing an order through payment, fulfillment, delivery, and — where applicable — returns and refunds.

State Transition Table

Flipkart Order — State Transition Table

Current State	Event / Trigger	Next State
Order Placed	Payment successful	Order Confirmed
Order Placed	Payment failed	Order Failed
Order Confirmed	Seller accepts & packs	Packed
Order Confirmed	Seller rejects / out of stock	Cancelled
Order Confirmed	User cancels	Cancelled
Packed	Handed to courier	Shipped
Packed	User cancels	Cancelled
Shipped	Reaches destination hub	Out for Delivery
Shipped	Lost/damaged in transit	Exception / Lost
Out for Delivery	Delivered to customer	Delivered
Out for Delivery	Delivery attempt failed	Delivery Failed / Re-attempt
Delivery Failed	Re-attempt scheduled	Out for Delivery
Delivery Failed	Max attempts exceeded	RTO (Return to Origin)
Delivered	Return window expires	Order Closed
Delivered	Customer initiates return	Return Requested
Return Requested	Pickup successful & QC passed	Return Approved
Return Requested	QC failed	Return Rejected
Return Approved	Refund/replacement processed	Refunded / Replaced
Refunded / Replaced	—	Order Closed
Return Rejected	—	Order Closed
Cancelled	—	Order Closed
RTO	—	Order Closed
Order Failed	—	Order Closed

Fig: State Transition Table

State Transition Diagram

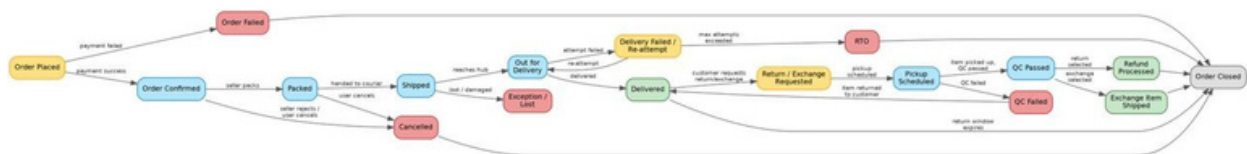


Fig: State Transition Diagram

Explanation

The table starts at **Order Placed**, branching to **Order Confirmed** on successful payment or **Order Failed** on payment failure. From there, the order moves through **Packed** → **Shipped** → **Out for Delivery** → **Delivered**, with cancellation possible at several earlier stages (seller rejection, out of stock, or user-initiated cancellation) and an **Exception/Lost** path if the shipment is lost or damaged in transit. Failed delivery attempts loop back through **Delivery Failed / Re-attempt** until either a successful delivery or **RTO (Return to Origin)** after max attempts are exceeded. Post-delivery, the customer can initiate a return, which goes through pickup, quality check (QC), and ends in either **Refunded/Replaced** or **Return Rejected**. Every

terminal state ultimately funnels into **Order Closed**, which keeps the model tidy and ensures no state is left dangling.

2. Naukri.com — Job Application Lifecycle

Naukri.com's application flow model shows a candidate's job application progresses from submission through recruiter screening, interviews, and final hiring decision.

State Transition Table

Naukri.com Job Application — State Transition Table

Current State	Event / Trigger	Next State
Application Submitted	Recruiter opens profile	Under Review
Application Submitted	Candidate withdraws	Withdrawn by Candidate
Under Review	Recruiter shortlists	Shortlisted
Under Review	Recruiter rejects	Not Shortlisted
Shortlisted	Interview slot fixed	Interview Scheduled
Shortlisted	Candidate withdraws	Withdrawn by Candidate
Interview Scheduled	Reschedule request	Interview Rescheduled
Interview Rescheduled	New slot confirmed	Interview Scheduled
Interview Scheduled	Interview held	Interview Completed
Interview Completed	Candidate selected	Offer Extended
Interview Completed	Candidate not selected	Rejected
Interview Completed	Next round scheduled	Interview Scheduled
Offer Extended	Candidate accepts	Offer Accepted
Offer Extended	Candidate declines	Offer Declined
Offer Accepted	Joining date confirmed	Hired / Onboarded
Not Shortlisted	—	Application Closed
Rejected	—	Application Closed
Offer Declined	—	Application Closed
Withdrawn by Candidate	—	Application Closed
Hired / Onboarded	—	Application Closed

Fig: State Transition Table

State Transition Diagram

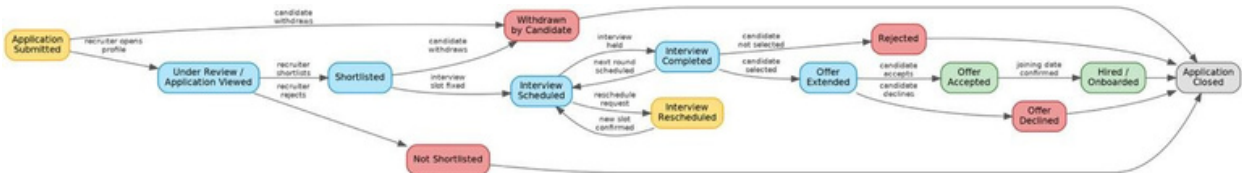


Fig: State Transition Diagram

Explanation

The application starts at **Application Submitted** and moves to **Under Review** once a recruiter opens the profile, or directly to **Withdrawn by Candidate** if the candidate pulls out early. If shortlisted, the flow proceeds to **Interview Scheduled**, with a **Interview Rescheduled** loop available if either party requests a new slot. After the interview is held, the outcome is either another round (looping back to **Interview Scheduled**), **Rejected**, or an **Offer Extended**. From there, the candidate's decision leads to either **Offer Accepted** (which becomes **Hired/Onboarded** once the joining date is confirmed) or **Offer Declined**. As with the other tables, every terminal outcome converges into a single **Application Closed** state.

3. Facebook — Friend Request Lifecycle

Facebook's friend-connection flow models the various relationship states two users can have, from strangers to friends, including restrictions, blocking, and unfriending.

State Transition Table

Facebook Friend Request — State Transition Table

Current State	Event / Trigger	Next State
No Relation / Strangers	User sends friend request	Friend Request Sent
Friend Request Sent	Delivered to recipient	Pending (Received)
Friend Request Sent	Sender cancels request	Request Cancelled
Request Cancelled	—	No Relation / Strangers
Pending (Received)	Recipient accepts	Friends
Pending (Received)	Recipient declines	Request Declined
Request Declined	—	No Relation / Strangers
Friends	Either user unfriends	Unfriended
Friends	User restricts other	Restricted
Friends	Either user blocks	Blocked
Unfriended	—	No Relation / Strangers
Restricted	Restriction removed	Friends
No Relation / Strangers	User blocks stranger	Blocked
Blocked	Blocker unblocks	Unblocked
Unblocked	—	No Relation / Strangers

Fig: State Transition Table

State Transition Diagram

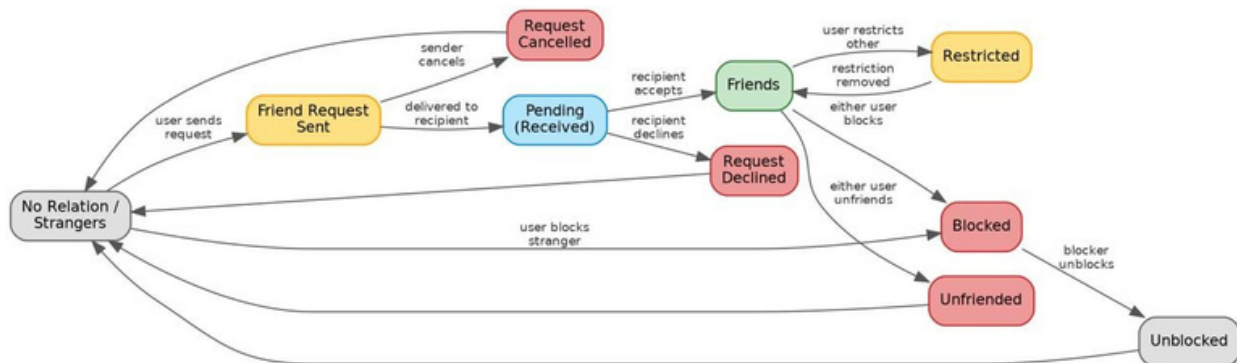


Fig: State Transition Diagram

Explanation

Two strangers begin in **No Relation/Strangers**. When one user sends a request, it moves to **Friend Request Sent**, then **Pending (Received)** once delivered. The recipient can accept (leading to **Friends**) or decline (leading to **Request Declined**, which resets back to strangers), while the sender can also cancel the request before it's actioned. Once **Friends**, users can apply a **Restricted** status on each other (reversible), fully **Unfriend** (which resets to strangers), or **Block** each other — a state reachable both from Friends and directly from a stranger relationship. Blocking is reversible via **Unblocked**, which returns the pair to **No Relation/Strangers**, closing the loop of the entire model.

4. Myntra — Order Lifecycle

Myntra's order flow closely mirrors Flipkart's, since both are e-commerce platforms, but places extra emphasis on the return/exchange path — reflecting fashion retail's high return rates.

State Transition Table

Myntra Order — State Transition Table

Current State	Event / Trigger	Next State
Order Placed	Payment successful	Order Confirmed
Order Placed	Payment failed	Order Failed
Order Confirmed	Seller accepts & packs	Packed
Order Confirmed	Seller rejects / user cancels	Cancelled
Packed	Handed to courier	Shipped
Packed	User cancels	Cancelled
Shipped	Reaches destination hub	Out for Delivery
Shipped	Lost/damaged in transit	Exception / Lost
Out for Delivery	Delivered to customer	Delivered
Out for Delivery	Delivery attempt failed	Delivery Failed / Re-attempt
Delivery Failed	Re-attempt scheduled	Out for Delivery
Delivery Failed	Max attempts exceeded	RTO
Delivered	Return window expires	Order Closed
Delivered	Customer requests return/exchange	Return / Exchange Requested
Return / Exchange Requested	Pickup scheduled	Pickup Scheduled
Pickup Scheduled	Item picked up, QC passed	QC Passed
Pickup Scheduled	QC failed	QC Failed
QC Passed	Return selected	Refund Processed
QC Passed	Exchange selected	Exchange Item Shipped
QC Failed	Item returned to customer	Delivered
Refund Processed	—	Order Closed
Exchange Item Shipped	—	Order Closed
Cancelled	—	Order Closed
RTO	—	Order Closed
Order Failed	—	Order Closed

Fig: State Transition Table

State Transition Diagram

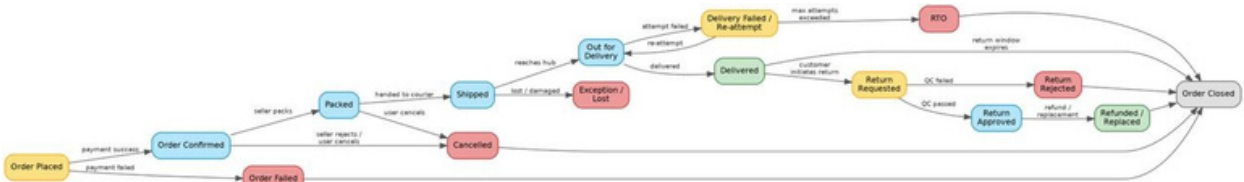


Fig: State Transition Diagram

Explanation

Like Flipkart, the order starts at **Order Placed**, and progresses through **Confirmed** → **Packed** → **Shipped** → **Out for Delivery** → **Delivered**, with cancellation and lost-in-transit exception paths along the way. The key difference appears after delivery: instead of a single "Return Requested" branch, Myntra explicitly separates **Return / Exchange Requested**, followed by **Pickup Scheduled** and a QC step that leads to either **QC Passed** or **QC Failed**. Critically, when QC passes, the customer's original choice determines the

outcome — **Refund Processed** if a return was selected, or **Exchange Item Shipped** if an exchange was selected — while a QC failure sends the original item back to the customer, returning the order to a **Delivered** state. All paths again converge to **Order Closed**.

5. Swiggy — Food Order Lifecycle

Swiggy's orderflow models a food-delivery order, which—unlike e-commerce — is time-sensitive and involves a restaurant partner and a delivery partner as additional actors.

State Transition Table

Swiggy Order — State Transition Table

Current State	Event / Trigger	Next State
Order Placed	Payment successful	Order Confirmed
Order Placed	Payment failed	Payment Failed
Payment Failed	—	Order Closed
Order Confirmed	Restaurant accepts	Food Being Prepared
Order Confirmed	Restaurant rejects	Rejected by Restaurant
Order Confirmed	User cancels (grace period)	Cancelled
Food Being Prepared	Food ready	Ready for Pickup
Food Being Prepared	User/support cancels	Cancelled
Ready for Pickup	Partner assigned	Delivery Partner Assigned
Delivery Partner Assigned	Partner picks up order	Order Picked Up
Order Picked Up	En route to customer	Out for Delivery
Out for Delivery	Delivered	Delivered
Out for Delivery	Customer unreachable	Delivery Failed / Unreachable
Delivery Failed / Unreachable	Order returned	Refund Initiated
Rejected by Restaurant	—	Refund Initiated
Cancelled	If payment already made	Refund Initiated
Cancelled	If not charged	Order Closed
Refund Initiated	Refund processed	Refunded
Refunded	—	Order Closed
Delivered	—	Order Closed

Fig: State Transition Table

State Transition Diagram

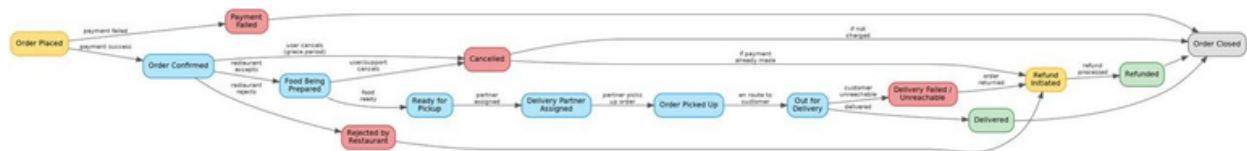


Fig: State Transition Diagram

Explanation

The order begins at **Order Placed**, moving to **Order Confirmed** on successful payment or **Payment Failed** otherwise. Once confirmed, the restaurant can accept (moving to **Food Being Prepared**) or reject the order (**Rejected by Restaurant**), and the user can still cancel during a short grace period. After preparation, the order proceeds through **Ready for Pickup** → **Delivery Partner Assigned** → **Order Picked Up** → **Out for Delivery**, ending in either **Delivered** or, if the customer is unreachable, **Delivery Failed/Unreachable** (which triggers a refund). Every cancellation or rejection path also funnels into **Refund Initiated** if payment was already collected, which then transitions to **Refunded**. As with the other examples, all terminal states ultimately reach **Order Closed**.